WK 3 ASSIGNMENT

Part 1: Theoretical Understanding

1. Short Answer Questions

***Q1**: Explain the primary differences between TensorFlow and PyTorch. When would you choose one over the other?*

Feature	**PyTorch**	**TensorFlow**
Debugging	Easier due to the dynamic graph and standard Python debugger compatibility.	The original static graph approach was more challenging.
High-Level API	Its native interface is generally considered lower-level.	Keras is the official high-level API, offering a simpler, consistent way to build and train models.
Community Focus	Highly popular in academia and research due to its flexibility.	Dominant in industry and enterprise applications due to its robust deployment features and Google backing.

Choose PyTorch when:

- You prioritise research, rapid prototyping, and experimentation. The "Define-by-Run" dynamic graph makes it easier to build custom, complex, or recurrent models with variable-length inputs and to try out new ideas quickly.
- You prefer a Pythonic and highly integrated coding style that is easier to read and debug using familiar Python tools.
- You are primarily working within the academic community, where PyTorch is now the dominant choice for publications and cutting-edge work.

Choose TensorFlow when:

- Your main goal is production deployment and scalability. TensorFlow's mature tools like TensorFlow Serving, TensorFlow Lite, and TensorFlow.js provide a more complete and tested end-to-end ML pipeline for various platforms.

- You need a framework with a vast, established industrial ecosystem and a high-level API (Keras) that simplifies development for general-purpose models.
- You are collaborating with a large enterprise team that already uses the TensorFlow ecosystem.

Q2: Describe two use cases for Jupyter Notebooks in AI development.

1. **Exploratory Data Analysis (EDA) and Data Preparation**

   Notebooks are the primary tool for the data phase. They allow developers to quickly load data with Pandas, check statistics, and instantly generate inline visualizations (e.g., histograms with Matplotlib) to understand distributions, identify outliers, and document the entire data cleaning and feature engineering pipeline transparently.

2. **Model Prototyping and Experimentation**

   They serve as an ideal "scratchpad" for modeling. Developers can build a model, train it, and instantly evaluate metrics (like loss and accuracy) cell by cell. This iterative workflow supports rapid hyperparameter tuning and ensures the final notebook is a reproducible research log of the exact code, parameters, and results of an experiment.

Q3: How does spaCy enhance NLP tasks compared to basic Python string operations?

Feature	**Basic Python String Operations**	**spaCy**
Tokenization	Split by whitespace or punctuation (crude).	Splits text based on linguistic rules (e.g., handles contractions like "don't" correctly).
Information Extraction	Requires custom RegEx for patterns like dates or emails.	Built-in models for Named Entity Recognition (NER) (e.g., identifies "Apple" as an *ORG*).
Linguistic Analysis	Cannot identify parts of speech or sentence structure.	Automatically provides POS Tags (e.g., noun, verb), lemmas, and dependency relationships.
Performance	Generally faster for simple find/replace.	Highly optimised and faster for large-scale, complex

		NLP tasks.

2. Comparative Analysis between Scikit-learn and TensorFlow

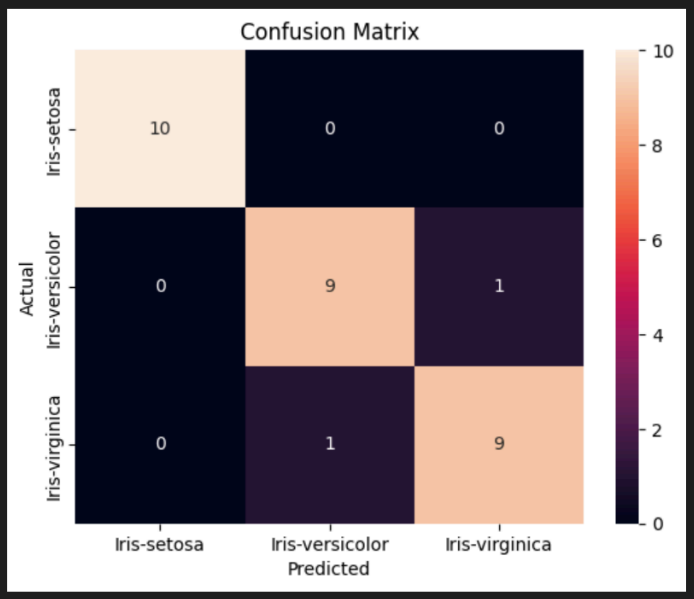
Aspect	Scikit-learn	TensorFlow
Target applications	**Used for** standard, structured data problems. It's built for **traditional ML** on organized data.	This is for problems involving massive, unstructured data that require complex layers of logic, like vision and language.
Ease of use for beginners	Very Easy. It’s the perfect starting point. Algorithms are pre-built and follow a single, simple pattern, so you can focus entirely on understanding ML concepts, not coding intricate network math.	Steeper Curve. While the high-level Keras API makes it much easier now, you're still building custom neural network architectures layer-by-layer. It demands you understand the *internals* of a deep learning model.
Community support	Strong & Academic. It has fantastic, clean documentation and is deeply integrated with the classic Python data science stack (Pandas, NumPy). It’s the most trusted library in many university courses and research papers.	Massive & Industry-Focused. Backed by Google, its ecosystem is vast, featuring powerful tools (TensorFlow Serving, TensorBoard) for large-scale production, distributed computing on GPUs, and mobile/web deployment.

Part 2: Practical Implementation

Task 1: *Classical ML with Scikit-learn*

```
==================================================
IRIS CLASSIFICATION PIPELINE
==================================================
Downloading Iris dataset from Kaggle...
Dataset URL: https://www.kaggle.com/datasets/uciml/iris
Dataset downloaded successfully!

==================================================
DATASET OVERVIEW
==================================================

Shape: (150, 6)

Columns: ['Id', 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm', 'Species']

First few rows:
   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm      Species
0   1            5.1           3.5            1.4           0.2  Iris-setosa
1   2            4.9           3.0            1.4           0.2  Iris-setosa
2   3            4.7           3.2            1.3           0.2  Iris-setosa
3   4            4.6           3.1            1.5           0.2  Iris-setosa
4   5            5.0           3.6            1.4           0.2  Iris-setosa

Data types:
...
    accuracy                           0.93        30
   macro avg       0.93      0.93      0.93        30
weighted avg       0.93      0.93      0.93        30
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...




```
==================================================
PER-CLASS METRICS
==================================================

Iris-setosa:
  Precision: 1.0000
  Recall:    1.0000

Iris-versicolor:
  Precision: 0.9000
  Recall:    0.9000

Iris-virginica:
  Precision: 0.9000
  Recall:    0.9000

==================================================
PIPELINE COMPLETED SUCCESSFULLY!
==================================================
```

Task 2: *Deep Learning with TensorFlow/PyTorch*

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 28, 28, 32)	320
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_4 (Conv2D)	(None, 14, 14, 64)	18,496
max_pooling2d_3 (MaxPooling2D)	(None, 7, 7, 64)	0
conv2d_5 (Conv2D)	(None, 7, 7, 64)	36,928
flatten_1 (Flatten)	(None, 3136)	0
dense_2 (Dense)	(None, 128)	401,536
dropout_1 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 10)	1,290

Total params: 458,570 (1.75 MB)

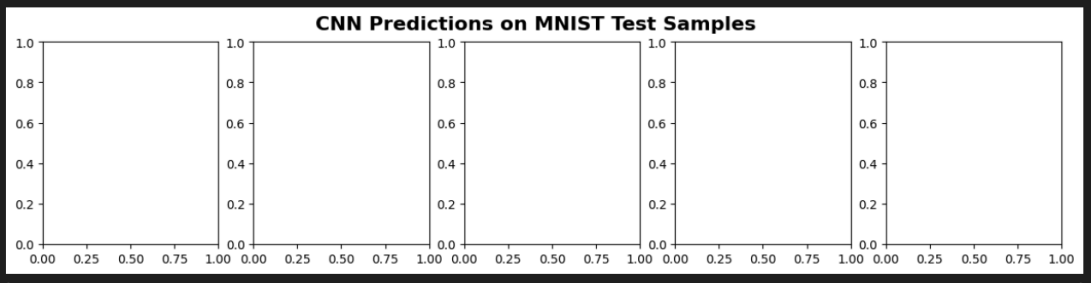
Trainable params: 458,570 (1.75 MB)

Non-trainable params: 0 (0.00 B)

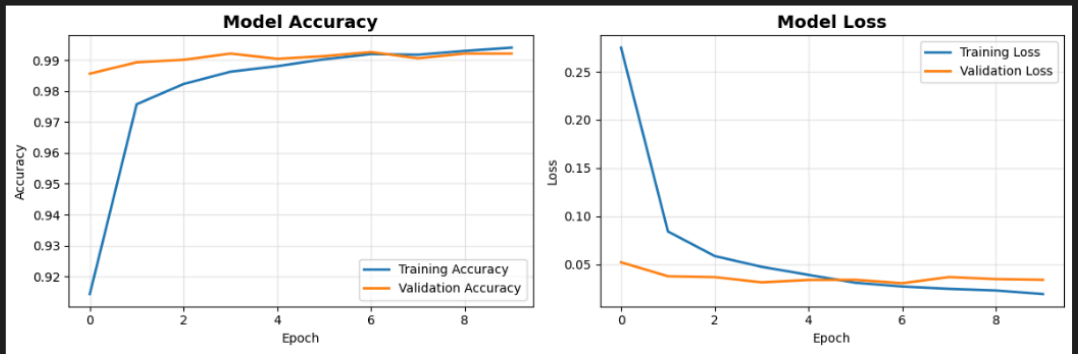
```
Training model...
Epoch 1/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 31s 69ms/step - accuracy: 0.9143 - loss: 0.2750 - val_accuracy: 0.9857 - val_loss: 0.0521
Epoch 2/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 28s 66ms/step - accuracy: 0.9758 - loss: 0.0842 - val_accuracy: 0.9893 - val_loss: 0.0377
Epoch 3/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 32s 77ms/step - accuracy: 0.9823 - loss: 0.0586 - val_accuracy: 0.9902 - val_loss: 0.0367
Epoch 4/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 43s 102ms/step - accuracy: 0.9863 - loss: 0.0474 - val_accuracy: 0.9922 - val_loss: 0.0313
Epoch 5/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 62s 55ms/step - accuracy: 0.9881 - loss: 0.0391 - val_accuracy: 0.9905 - val_loss: 0.0338
Epoch 6/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 29s 70ms/step - accuracy: 0.9903 - loss: 0.0309 - val_accuracy: 0.9913 - val_loss: 0.0339
Epoch 7/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 44s 103ms/step - accuracy: 0.9920 - loss: 0.0270 - val_accuracy: 0.9927 - val_loss: 0.0304
Epoch 8/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 49s 116ms/step - accuracy: 0.9918 - loss: 0.0246 - val_accuracy: 0.9907 - val_loss: 0.0368
Epoch 9/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 62s 146ms/step - accuracy: 0.9931 - loss: 0.0229 - val_accuracy: 0.9922 - val_loss: 0.0347
Epoch 10/10
422/422 ━━━━━━━━━━━━━━━━━━━━ 44s 104ms/step - accuracy: 0.9941 - loss: 0.0191 - val_accuracy: 0.9922 - val_loss: 0.0339
```

```
Evaluating model on test set...

Test Loss: 0.0246
Test Accuracy: 99.10%
✓ Target accuracy of 95% achieved!
```

```
Generating predictions on 5 sample images...

Text(0.5, 0.98, 'CNN Predictions on MNIST Test Samples')
```




```
Visualization saved as 'mnist_predictions.png'
Training history saved as 'training_history.png'

==================================================
SUMMARY
==================================================
Model: CNN with 3 Conv layers
Total Parameters: 458,570
Training Samples: 60,000
Test Samples: 10,000
Final Test Accuracy: 99.10%
Status: SUCCESS ✓
==================================================

<Figure size 640x480 with 0 Axes>
```



Task 3: *NLP with spaCy*

```
================================================================================
AMAZON PRODUCT REVIEW ANALYSIS
================================================================================

Review #1
--------------------------------------------------------------------------------
Text: I absolutely love my new Apple iPhone! The camera quality is amazing and the battery life is excellent. Best phone I've ever owned.

Extracted Entities:
  Brands: Apple
  Products: The camera, Best phone

Sentiment Analysis:
  Overall: POSITIVE
  Confidence: 1.0
  Positive Score: 5.5
  Negative Score: 0

================================================================================

Review #2
--------------------------------------------------------------------------------
...
  Samsung earbuds: 1 mentions
  for book: 1 mentions
  running shoes: 1 mentions
================================================================================
```

Part 3: Ethics & Optimisation

1. Ethical Considerations

★ **Potential Biases in MNIST and Amazon Reviews Models**

- **MNIST Model (Handwritten Digit Recognition)**

MNIST can exhibit Data Collection and Style Biases:

- Style/Stroke Bias and Minority Representation Bias occur when the model performs poorly on handwriting variations not present in the original collector groups (high school students and U.S. Census Bureau employees), leading to poor generalization on rare digit styles.
- Spurious Correlation Bias makes the model rely on irrelevant features like digit color or background texture if accidentally correlated with a class.

- **Amazon Reviews Model (Sentiment/Topic Analysis)**

Sentiment analysis models are highly vulnerable to Societal and Lexical Biases:

- Identity Terms Bias creates spurious associations between identity terms (names, pronouns) and sentiment, potentially assigning unfairly lower scores to neutral text mentioning specific demographics.
- Class Imbalance Bias (skew toward positive reviews) biases the model toward predicting positive sentiment, making it less sensitive to genuinely negative reviews.
- Language and Slang Bias (Lexical) causes the model to misclassify non-standard language or linguistic variations as having a more negative or "toxic" sentiment, resulting in disparate impact.

★ **Mitigation with Specialized Tools**

- **TensorFlow Fairness Indicators (TFFI)**

TFFI is a post-training auditing tool that focuses on measuring and quantifying model performance across predefined groups ("slices") to identify disparities. It calculates metrics like False Positive Rate (FPR), False Negative Rate (FNR), Equal Opportunity, or Predictive Parity to show statistically significant differences. TFFI's results guide remediation (e.g., augmenting training data or adjusting the model's loss function) and support the generation of transparency reports and Model Cards.

- **spaCy’s Rule-Based Systems**

spaCy's rule-based systems are used for pre-processing and in-processing mitigation by directly modifying text data features. The Matcher component can neutralize identity bias by identifying and replacing identity terms with neutral placeholders (e.g., [PERSON_NAME]) before training. Custom components can normalize lexical variations (slang, non-standard language) to reduce toxicity/slang bias. Furthermore, Coreference Resolution correctly attributes sentiment expressed via pronouns to the intended entity, improving feature association and fairness.

2. Troubleshooting Challenge